

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

CSE-2340: Software Development 1

Flexbox, Grid & Responsive Design

Week 3 · Class 1 (teaching) + Class 2 (practice) · Mobile-first layout

Md Sadman Hafiz

Lecturer, Dept. of CSE, IIUC

Autumn 2026 | two 50-minute classes

Recall & Today's Outcome

? Recall from Week 2

- What does box-sizing: border-box change?
- Why does width do nothing on a span?
- How do you centre a block with a max-width?
- Remember the Open button in Exercise 4?

✓ By the end of today you can

- Lay out a row or column with Flexbox
- Build a two-dimensional layout with Grid
- Choose correctly between the two
- Write mobile-first media queries
- Make a page work from 360px to 1440px
- Inspect and debug layout in DevTools

The Problem Flexbox Solves

LAST WEEK — the hard way

```
.card {  
  position: relative;  
  padding-right: 120px;  
}  
  
.btn {  
  position: absolute;  
  right: 20px;  
  top: 50%;  
  /* and it still breaks */  
}
```

TODAY — the same result

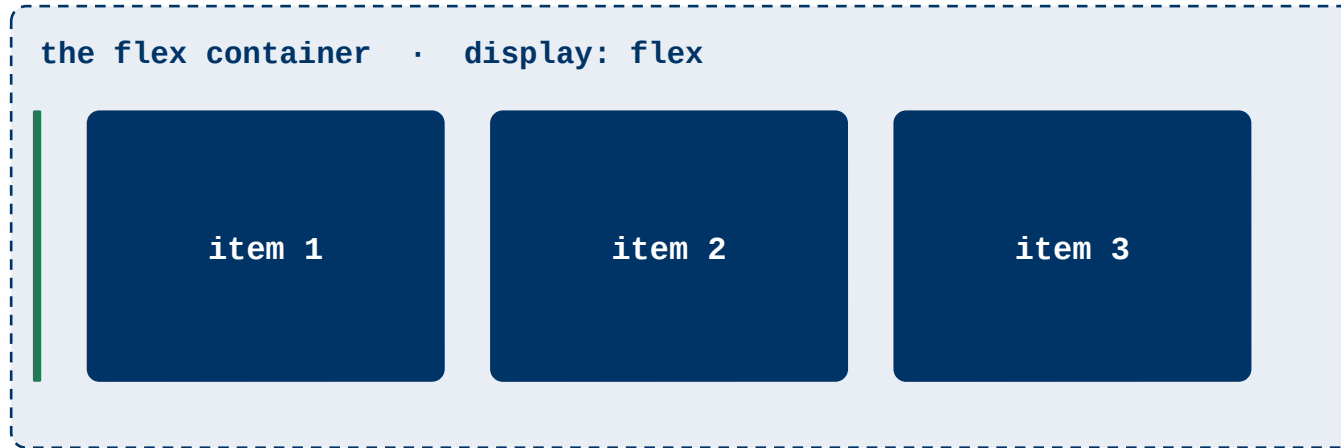
```
.card {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```



Flexbox and Grid exist because the box model alone cannot arrange things

- The box model tells one element how much space to take. It says nothing about how several elements sit next to each other.
- Before 2015 people used floats and positioning for this, which is why old tutorials look so strange.
- Flexbox arranges things in ONE direction. Grid arranges them in TWO.

Flexbox: Container and Items



main axis → justify-content controls this

cross axis ↓ align-items controls this

display: flex on a parent immediately puts its direct children in a row. That one line is 80% of Flexbox.

Two sets of properties

- On the CONTAINER: display, flex-direction, justify-content, align-items, gap, flex-wrap
- On the ITEMS: flex, flex-grow, flex-basis, align-self

! The rule that prevents confusion

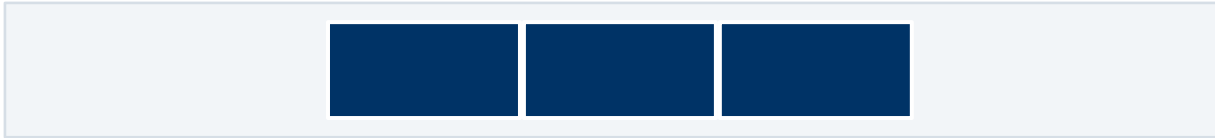
- The main axis follows flex-direction
- Change direction and justify-content changes meaning too

justify-content: Along the Main Axis

flex-start



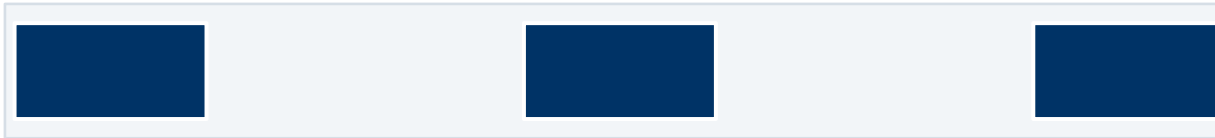
center



flex-end



space-between



space-around



```
.row {  
  display: flex;  
  justify-content:  
    space-between;  
}
```

Use it for

- A logo left, menu right: space-between
- Centring a row of buttons: center
- Even spacing: space-around

align-items, gap and flex-direction

```
.row {  
  display: flex;  
  flex-direction: row;      /* or column */  
  justify-content: space-between;  
  align-items: center;      /* cross axis */  
  gap: 16px;                /* space BETWEEN items */  
  flex-wrap: wrap;         /* allow a second line */  
}
```



Vertical centring, finally

display: flex plus align-items: center. Two lines. This used to be the hardest thing in CSS.

align-items: stretch

Default. Items fill the container height

align-items: flex-start

Pinned to the top

flex-wrap: wrap

Items drop to a new line instead of squashing

align-items: center

Vertically centred — the classic use

gap: 16px

Space between items only, not on the outside

flex-direction: column

Main axis becomes vertical — everything rotates

Sizing Items: the flex Shorthand

```
/* Each item takes an equal share */
.item {
  flex: 1;
}

/* One item takes twice the space */
.sidebar { flex: 1; }
.content { flex: 2; }

/* Do not grow, do not shrink, stay 240px */
.fixed {
  flex: 0 0 240px;
}
```

Rule of thumb

Container properties decide the arrangement. Item properties decide how leftover space is divided. If items are the wrong size, look at the item; if they are in the wrong place, look at the container.

flex: 1 means

- grow to fill leftover space
- shrink if there is not enough
- starting from a base of 0

! This is what you will use

- flex: 1 on cards makes them equal width
- flex: 2 on one makes it double
- You rarely need flex-grow and flex-basis separately

Live Build: Nav Bar and Card Row

```
/* 1. The nav bar: title on the left, links on the right */
header {
  display: flex;
  justify-content: space-between; /* pushes them apart */
  align-items: center;           /* same vertical line */
  flex-wrap: wrap;
  gap: 12px;
}

nav ul {
  display: flex;
  gap: 20px;
  list-style: none;
}

/* 2. The card row: three equal cards that wrap on a small screen */
.card-row {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.card {
  flex: 1 1 280px; /* grow, shrink, never narrower than 280px */
}
```


01

CSS Grid

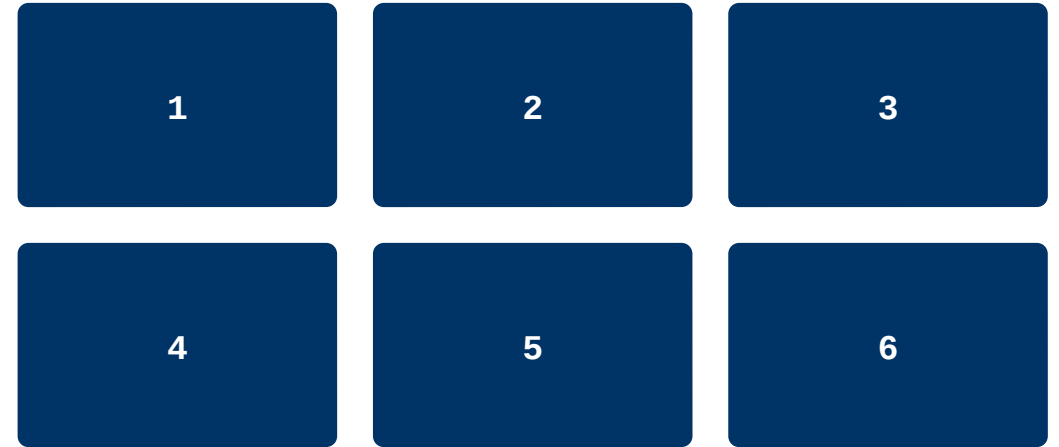
Flexbox arranges in one direction. Grid arranges in two at once.

Grid: Columns and Rows

```
.gallery {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
}  
  
/* repeat() says the same thing */  
.gallery {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}
```

! The fr unit

- fr means one share of the leftover space
- 1fr 1fr 1fr = three equal columns
- 2fr 1fr = the first column is twice as wide
- Unlike %, fr already accounts for the gap



repeat(3, 1fr) with gap: 20px

Two lines is a grid

display: grid and grid-template-columns. Everything else is refinement.

The One Grid Line Worth Memorising

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));  
  gap: 20px;  
}
```

1 auto-fit

Fit as many columns as will comfortably fit

2 minmax(260px, 1fr)

Never narrower than 260px, otherwise share the space equally

3 The result

4 columns on a laptop, 2 on a tablet, 1 on a phone



This is responsive with no media query at all

Resize the browser and the column count changes by itself. For galleries, card lists and product grids, this single declaration is usually the whole job.

Flexbox or Grid?

Use Flexbox when

- The items sit in ONE line or column
- A nav bar: title left, links right
- A row of buttons
- Vertically centring something
- The content decides the sizes

Use Grid when

- You need rows AND columns at once
- A gallery or card grid
- A whole page skeleton
- You want the layout to decide the sizes



You will use both on the same page, and that is correct

- Typical structure: Grid for the page skeleton, Flexbox inside each area for its contents.
- If you cannot decide, ask: am I arranging along one line, or filling a table of slots?

02

Responsive & Mobile-First

Most of your users are on a phone. Design for that first, then add the desktop.

Mobile-First: Why and How

```
<!-- Without this, a phone pretends to be
      980px wide and shrinks everything -->
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

DESKTOP-FIRST — harder

```
.card-row { display: flex; }

@media (max-width: 768px) {
  .card-row { display: block; }
}
```

MOBILE-FIRST — do this

```
/* base = phone, no query */
.card-row {
  display: block;
}

/* then ADD for bigger screens */
@media (min-width: 768px) {
  .card-row {
    display: flex;
    gap: 20px;
  }
}
```

! Why mobile-first wins

- The simplest layout needs no query at all
- You add complexity instead of undoing it
- Old phones never download rules they cannot use

Media Queries and Breakpoints

```
/* Base styles: phone. No media query. */
.container { padding: 16px; }

/* Tablet and up */
@media (min-width: 768px) {
  .container { padding: 24px; }
  .card-row { display: flex; gap: 20px; }
}

/* Laptop and up */
@media (min-width: 1024px) {
  .container { max-width: 1100px; }
}
```

Test it properly

- DevTools: click the phone icon (Ctrl+Shift+M)
- Try 360px, 768px and 1280px
- Drag the edge slowly and watch where it breaks

Phone

base, no query

up to 767px

Tablet

min-width: 768px

768 - 1023px

Laptop

min-width: 1024px

1024px and up



Two breakpoints is enough

You do not need one per device. Add a breakpoint where the layout actually breaks, not at a famous number.

Live Build: A Responsive Page

```
/* ----- BASE: phone. Everything stacks. ----- */
* { box-sizing: border-box; }

.container { width: 92%; max-width: 1100px; margin: 0 auto; padding: 16px; }

header { display: flex; flex-direction: column; gap: 8px; }

.gallery { display: grid; gap: 16px; }          /* one column by default */

/* ----- TABLET AND UP ----- */
@media (min-width: 768px) {
  header { flex-direction: row; justify-content: space-between; align-items: center; }

  .gallery { grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); }
}

/* ----- LAPTOP AND UP ----- */
@media (min-width: 1024px) {
  .container { padding: 24px; }
}
```


Common Mistakes

Flexbox does nothing

display: flex must be on the PARENT, not the items.

justify-content is not working

You wanted align-items. Check which axis you mean.

Items squash instead of wrapping

Add flex-wrap: wrap. It is not on by default.

The grid has one column only

You set display: grid but no grid-template-columns.

The phone layout looks zoomed out

Missing the viewport meta tag in the head.

My media query is ignored

It sits above the base rule, or the syntax is (min-width: 768px) with the colon.

Order matters in CSS: a later rule wins over an earlier one with the same specificity. Media queries go at the BOTTOM.

03

Class 2 — Practice

Open the starter folder. You build and debug; I circulate.

Class 2: What You Will Do

1 Fix the broken layout

- Eight bugs planted in the CSS
- 12 min

2 Grid gallery

- auto-fit and minmax from scratch
- 8 min

3 Make it responsive

- Mobile-first, two breakpoints
- 8 min

4 Rebuild the target

- Stretch task, and MP1 practice
- 7 min

! Mini Project 1 is evaluated in C4 this week

Everything you do today is directly reusable in it. Exercise 4 is deliberately close to what MP1 asks for.

DevTools for Layout

The flex and grid badges

In the Elements panel, a small 'flex' or 'grid' badge appears next to any container. Click it.

Overlay lines

Clicking the badge draws the axes, the gaps and the grid lines directly on the page.

Device mode

Ctrl+Shift+M. Test at 360px, 768px and 1280px.

The Styles panel

Media queries that do not apply are greyed out — a fast way to see which one is active.



Same four-step method as last week

Inspect → read the overlay → find the rule → test live. The only difference is that you are now reading axes and tracks instead of the box model.

Exercise 1: Fix the Broken Layout

Files: starter/index.html and starter/styles.css. Eight bugs. The HTML is correct — do not change it.

1 The nav items are still stacked

Which element needs display: flex?

3 The links sit above the title, not beside it

Cross axis this time

5 There is no space between cards

Do not reach for margin

7 Columns do not change on resize

The auto-fit line from C1

2 Title and links are jammed together

Which axis, and which property?

4 Cards squash instead of wrapping

One missing declaration

6 The gallery is a single column

display: grid alone is not enough

8 The phone layout is tiny and zoomed out

Look in the head, not the CSS

Exercise 2: Grid Gallery

```
<section class="gallery">
  <article class="tile">Week 1</article>
  <article class="tile">Week 2</article>
  <article class="tile">Week 3</article>
  <article class="tile">Week 4</article>
  <article class="tile">Week 5</article>
  <article class="tile">Week 6</article>
</section>
```

! Then answer in a CSS comment

- How many columns do you see at 1280px? At 800px? At 375px?
- What happens if you change 240px to 400px?
- Why does this need no media query?

Build it so that

- Tiles are never narrower than 240px
- The column count changes by itself on resize
- There is a 20px gap in both directions
- No media query is used at all

✓ Done when

Dragging the browser edge changes the column count smoothly, with no jumps or overflow

Exercise 3: Make It Responsive

Required behaviour

- Phone (360px): everything in one column, header stacked
- Tablet (768px): header becomes a row, cards in two columns
- Laptop (1280px): container capped, cards in three or four columns



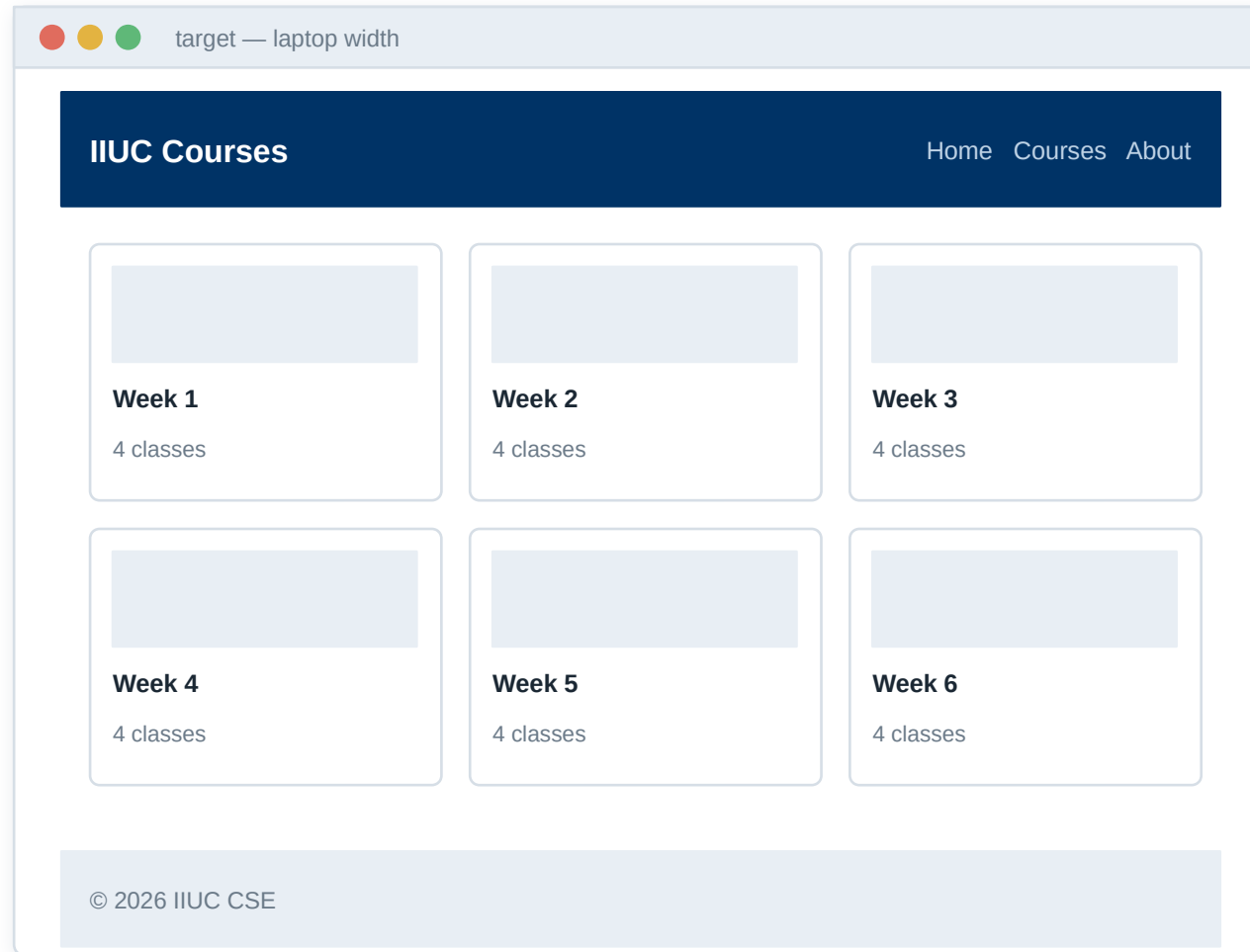
Rules

- Mobile-first: the base styles have NO media query
- Maximum two breakpoints
- Use min-width, never max-width

```
/* Start here. Base = phone. Then add. */
header { display: flex; flex-direction: column; gap: 8px; }

@media (min-width: 768px) {
  /* your job */
}
```

Exercise 4: Rebuild the Target



Requirements

- Flexbox for the header
- Grid for the six cards
- Three columns on a laptop, two on a tablet, one on a phone
- Semantic HTML, written by you

! This is Mini Project 1

- MP1 wants exactly this: semantic HTML, CSS, responsive layout, deployed
- Finish this well and MP1 is mostly done

Bug Clinic

Flexbox is doing nothing

display: flex is on the child. It belongs on the parent.

Everything is centred vertically but not horizontally

You swapped justify-content and align-items. Check the axis.

Cards overflow the screen on a phone

A fixed width somewhere. Use flex-basis or minmax instead.

The grid ignores auto-fit

minmax is missing, or the container has no width to work with.

My breakpoint fires at the wrong size

You used max-width in a mobile-first stylesheet. Pick one direction.

It works at 1280px and breaks at 360px

You wrote desktop-first. Start from the phone and add upwards.

Before asking for help: open the flex or grid overlay and say which axis you think is wrong. That question answers itself most of the time.

Assignment - Week 3, C1 and C2

1 Part 1 - In-class demonstration

- Time: last 5 minutes of C2
- Task: show your fixed starter page with at least six of the eight bugs corrected
- Resize to 360px and 1280px in front of the instructor
- Open the flex or grid overlay and explain one fix
- Submission: show on your screen before leaving
- Marks: 2 (checked on the spot)

2 Part 2 - Home task

- Finish all eight fixes and Exercises 2, 3 and 4
- Make your Week 2 pages fully responsive: mobile-first, maximum two breakpoints
- Write layout-notes.md: where each breakpoint is and why you chose it
- Commit and push
- Deadline: before Week 3, Class 3 · Marks: 3

Mini Project 1 is due and evaluated in C4 this week. Repository link and deployed URL before the class.

Marking Rubric & Exit Check

RUBRIC - 5 MARKS TOTAL

Six or more of the eight planted bugs fixed **1.5**

Grid gallery uses auto-fit and minmax correctly **1.0**

Mobile-first, min-width only, at most two breakpoints **1.5**

Target page rebuilt with Flexbox and Grid together **1.0**

Exit question 1 - concept

Name one layout Flexbox handles better than Grid, and one the other way round.

Exit question 2 - practical

Write the one grid line that makes a gallery responsive with no media query.

Next class (C3): specificity, the cascade and DevTools inspection — why one rule beats another. Then MP1 evaluation in C4.